

FINAL INFORMATICA I Parte 2		Duración	Fecha	Hojas
		60 minutos	30/jul/2026	
Nombre y Apellido	Nº Legajo	Calificación final		Docente Evaluador
		número	letras	Nombre
				Firma

Numere las hojas entregadas, y complete el casillero **Hojas** con la cantidad.

Lea detenidamente el enunciado, su correcta interpretación forma parte de esta evaluación.

Requerimiento:

Desarrolle una aplicación servidora que atienda múltiples conexiones de clientes en forma concurrente.

Por cada conexión recibida, el servidor deberá crear un nuevo thread encargado de atender exclusivamente a ese cliente.

Cada thread deberá quedar a la espera de recibir una trama ASCII de hasta 1024 bytes. La trama se considerará finalizada al encontrarse el primer carácter nulo '\0'. Una vez recibida la trama completa, se deberán contar las apariciones de los caracteres 'a', como 'A'.

Si la cantidad total de apariciones de 'a' y 'A' sumados es, como mínimo, el doble de la cantidad de apariciones de cualquier otro carácter individual de la trama, el servidor deberá responder “Trama tipo A”, y en caso contrario, deberá responder “Trama tipo Unknow”.

Luego de enviar la respuesta, la conexión deberá cerrarse y el thread deberá finalizar.

Por su parte, si la trama recibida es “Reporte de estado”, el servidor no deberá responder al cliente. En este caso, deberá cerrar la conexión e imprimir en pantalla: La cantidad total de conexiones respondidas y La cantidad total de conexiones rechazadas.

Se considerará una conexión respondida cuando el servidor haya enviado las tramas "Trama tipo A" o "Unknow".

Se considerará una conexión rechazada cuando el servidor no la procese por haber alcanzado el máximo de threads simultáneos. Los de solicitud de reporte no se cuentan.

El servidor podrá mantener como máximo 10 threads de atención activos al mismo tiempo.

Si se recibe una nueva conexión cuando ya existen 10 threads activos, el servidor deberá: Enviar al cliente la trama “Indisponible”, cerrar inmediatamente la conexión y, obviamente, no crear un nuevo thread para atenderla.

La solución deberá contemplar la sincronización necesaria para proteger las variables compartidas entre threads, como también asegurar la correcta liberación de los recursos utilizados cuando corresponda.

Al ser un servidor, no se espera que finalice.

El puerto utilizado para recibir las conexiones es el 30001.

Listado de funciones útiles

La siguiente es una lista de funciones que les puede ser de utilidad.

Bajo ningún aspecto debe considerar ni que debe usar todas las funciones de esta lista, ni que no puede utilizar otras.

Funciones de la cátedra para manejo de sockets: (network.c y .h)

```
int OpenServer(int portnr);
```

vincula al proceso (server) con un puerto de la computadora vía SO

parámetro de entrada

portnr: Numero de puerto en el que se busca dar disponibilidad el servicio

valor de retorno:

-1: error en el proceso

En caso de éxito, un descriptor de archivo para el nuevo socket es devuelto

```
int CloseServer(int sockfd);
    Cierra el socket y libera el recurso
    parámetro de entrada
        sockfd: Valor obtenido mediante OpenServer (si este fue >0)

int WaitConnect (int msSock);
    Establece una nueva conexión con un cliente
    parámetro de entrada
        msSock: Valor de socket obtenido mediante OpenServer (si este fue >0)
    valor de retorno:
        -1: error en el proceso
        En caso de éxito retorna un valor mayor a cero, que podrá ser utilizado por recv/send

int CloseConnect(int sockfd);
    Cierra la conexión y libera el recurso
    parámetro de entrada
        sockfd: Valor de socket obtenido mediante WaitConnect (si este fue >0)
```

Hilos:

```
int pthread_create( pthread_t * thread, const pthread_attr_t * attr,
    void *(*start_routine)(void *), void * arg);
int pthread_join (pthread_t thread, void **retval);
int pthread_detach(pthread_t thread);
pthread_t pthread_self(void);
void pthread_exit (void *retval);
int pthread_mutex_init(pthread_mutex_t * mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

señales:

```
sighandler_t signal(int signum, sighandler_t handler);
```

Memoria dinámica:

```
void *malloc(size_t size);
void free(void *ptr);
```

Lectura / escritura de sockets:

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
también puede utilizar read() y write()
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
```

Funciones varias:

```
int strncmp(const char *s1, const char *s2, size_t n);
int strcmp(const char *s1, const char *s2);
int strlen (const char *str);
```